
Corporate Credit Card Misuse Detection Using Machine Learning and ETL Pipeline

Pooja Malakappa Nuchchi¹

Abstract

Corporate credit card misuse is a significant financial and compliance challenge in enterprise settings. This paper presents an end-to-end machine learning system for detecting fraudulent corporate card transactions, combining a full ETL pipeline (PostgreSQL, Apache Airflow) with a five-model comparison (Logistic Regression, LinearSVC, Random Forest, XGBoost, LightGBM). We use the ULB Credit Card Fraud Detection dataset (284,807 transactions, 0.17% fraud) and address severe class imbalance using SMOTE and `class_weight='balanced'`. Random Forest achieves the best performance with AUC-ROC 0.9845, Precision 0.86, and F1-score 0.84, producing only 13 false positives on the test set. Results are served through a FastAPI backend with six REST endpoints and visualized in a React dashboard.

1. Introduction and Problem Statement

Corporate credit card misuse is a significant financial and compliance challenge. Employees issued corporate cards for legitimate business expenses may use them for unauthorized personal purchases, resulting in financial losses and policy violations. Manual review of thousands of transactions is time-consuming and not scalable.

Formally, the learning problem is defined as: given a transaction record $x \in R^{30}$ consisting of 28 anonymized behavioral features $V_1, \dots, V_{28}$, a normalized transaction amount, and hour of day, predict a binary label $y \in \{0, 1\}$ where $y = 1$ indicates misuse and $y = 0$ indicates legitimate spending. The dataset used is the Credit Card Fraud Detection dataset (ULB, Kaggle) containing 284,807 transactions with 99.83% legitimate vs. 0.17% fraudulent — a severely imbalanced classification problem.

2. Motivation

Financial fraud detection is one of the most practically important applications of machine learning. Organizations

lose significant revenue to fraud annually, with corporate card misuse among the most common types. Automated detection systems reduce investigation time and improve accuracy compared to rule-based systems. Multiple studies have shown that models like Random Forest and Logistic Regression can effectively detect fraudulent transactions (Wang et al., 2023; Hande et al., 2019). The core challenge is severe class imbalance, which requires techniques like SMOTE oversampling and data augmentation (Khalid et al., 2024; Strelcenia & Prakoonwit, 2023).

3. Related Work

Logistic Regression is a common baseline for fraud classification because it is simple and interpretable. Wang et al. compared Logistic Regression and Random Forest on simulated transaction data and found Random Forest performs better (Wang et al., 2023). Sopiyan et al. also compared Logistic Regression with Random Forest and Gradient Boosting using SMOTE and found ensemble methods outperform the logistic baseline (Sopiyan et al., 2022).

Support Vector Machines have been used in fraud detection for finding decision boundaries in high-dimensional spaces. Xia et al. applied SVM with GridSearchCV tuning to the Kaggle credit card dataset, optimizing AUC and F1-score (Xia, 2022). Aftab et al. used SVM alongside Random Forest and Logistic Regression and found ensemble methods outperform single classifiers (Aftab et al., 2023).

Random Forest is one of the most effective models for fraud detection because its ensemble voting mechanism naturally handles class imbalance. Hande et al. compared Logistic Regression, Decision Tree, and Random Forest and found Random Forest performs best (Hande et al., 2019). Aburbeian and Ashqar combined Random Forest with SMOTE and achieved strong results on imbalanced credit card data (Aburbeian & Ashqar, 2023).

Gradient Boosting methods are the current state of the art for tabular fraud detection. XGBoost builds trees sequentially, each correcting the residual errors of the previous. LightGBM uses leaf-wise tree growth, which makes it faster on large datasets. Sopiyan et al. found Gradient Boosting competitive with Random Forest on credit card

fraud data (Sopiyan et al., 2022).

Class Imbalance Handling: SMOTE creates synthetic minority class samples by interpolating between existing ones. Khalid et al. tested ensemble methods with SMOTE on European credit card transactions and showed that re-sampling improves minority class detection (Khalid et al., 2024). Tran and Dang compared SMOTE and ADASYN for balancing credit card fraud data before training Random Forest and Logistic Regression (Tran & Dang, 2021). Strelcenia and Prakoonwit proposed K-CGAN as an alternative to SMOTE (Strelcenia & Prakoonwit, 2023).

Production ML Pipelines: Most prior studies focus only on model accuracy without considering deployment. ETL pipelines that separate ingestion, transformation, and loading ensure reproducibility and scalability. This project adopts that principle using PostgreSQL and Apache Airflow.

4. How This Approach Differs

Most prior work focuses only on model performance in isolation. This project combines a complete data engineering layer (ETL pipeline, PostgreSQL, Apache Airflow) with a five-model comparison (Logistic Regression, LinearSVC, Random Forest, XGBoost, LightGBM), a REST API backend (FastAPI), and an interactive React dashboard. This end-to-end production-style system addresses real-world deployment considerations that are typically missing from academic fraud detection papers. Figure 1 shows the full system architecture, from raw CSV ingestion through the ETL pipeline to trained model serving via FastAPI.

The ETL pipeline consists of three modular Python scripts — `src/extract.py`, `src/transform.py`, and `src/load.py` — orchestrated by an Apache Airflow DAG (`dag_fraud.py`). This separation of concerns ensures each stage can be tested, monitored, and rerun independently. Transformed data is stored in a PostgreSQL database (`corp_card_fraud`), from which the FastAPI backend serves six REST endpoints for real-time inference and result retrieval. The React dashboard consumes these endpoints to visualize predictions interactively.

5. Dataset

5.1. Input and Output

The input is a transaction record $x \in R^{30}$ with three types of features: (1) 28 PCA-transformed anonymized features $V_1, \dots, V_{28}$ representing behavioral patterns, (2) `Amount_Scaled`, the transaction amount standardized using z-score normalization:

$$z = \frac{x - \mu}{\sigma}$$

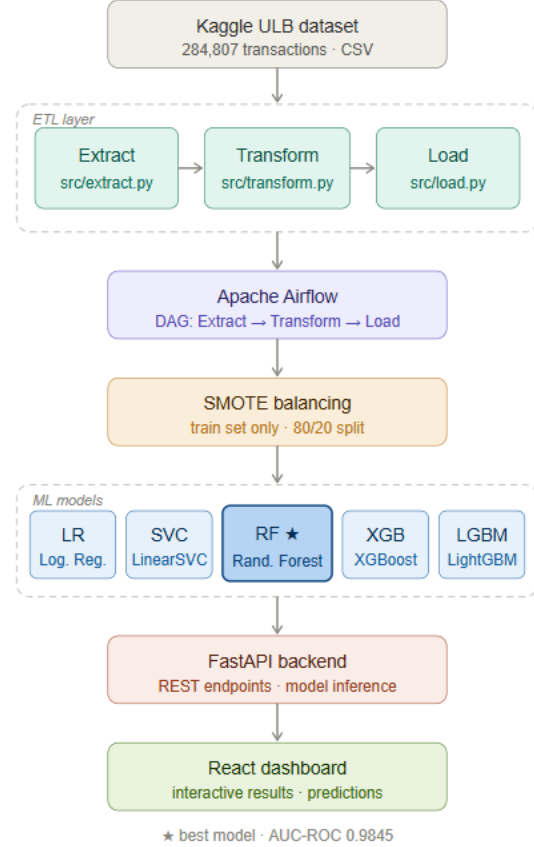


Figure 1. End-to-end system architecture. Raw transaction data flows through the ETL pipeline orchestrated by Airflow, is balanced with SMOTE, then passed to five ML models. Random Forest (★, AUC-ROC 0.9845) is served via FastAPI and visualized in a React dashboard.

and (3) `Hour`, the hour of day extracted from the raw `Time` field as:

$$\text{Hour} = \left\lfloor \frac{\text{Time}}{3600} \bmod 24 \right\rfloor$$

The output is a binary label $y \in \{0, 1\}$ where $y = 1$ indicates fraud.

5.2. Distribution, Size, and Missingness

The dataset contains 284,807 transactions. Figure 2 shows the severe class imbalance visually — 284,315 legitimate (99.83%) and 492 fraudulent (0.17%). The fraud class is nearly invisible compared to legitimate transactions. There are no missing values in any column. Most transaction amounts are small (under \$500 for both classes), but legitimate transactions have outliers up to \$25,691 while fraud transactions never exceed \$2,125 (Figure 3). Fraudulent transactions are more evenly spread across hours, with 2AM having the highest fraud rate at 1.713% (Figure 4).

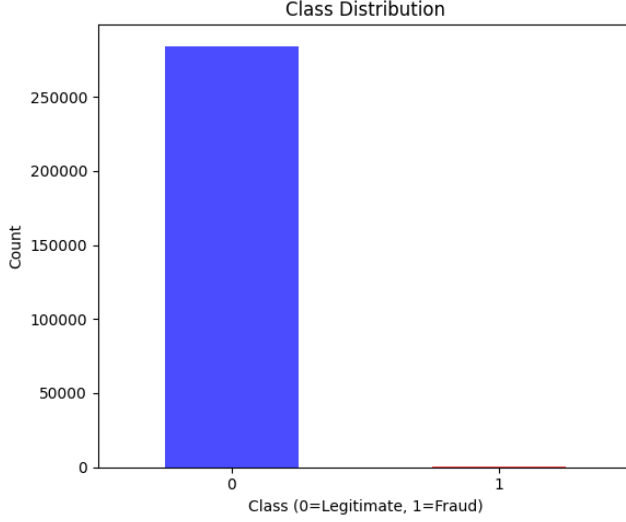


Figure 2. Class distribution showing severe imbalance — 99.83% legitimate vs. 0.17% fraudulent transactions.

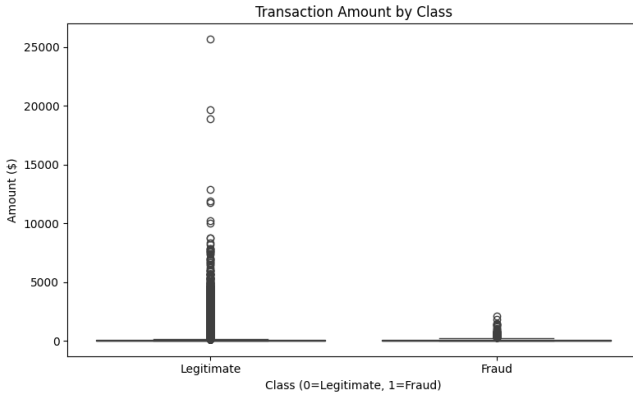


Figure 3. Transaction amount by class. Legitimate transactions have outliers up to \$25,691; fraud stays below \$2,125.

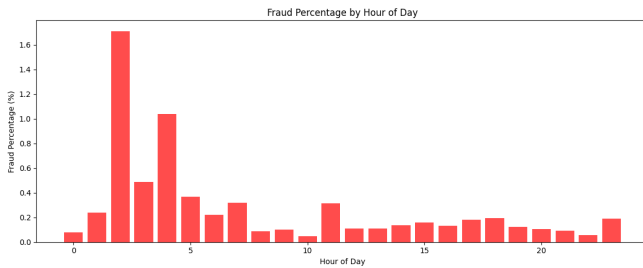


Figure 4. Fraud percentage by hour of day. 2AM has the highest fraud rate at 1.713%.

5.3. Preprocessing

The preprocessing pipeline (`src/transform.py`) performs three steps: (1) engineer the `Hour` feature from

the raw `Time` column, (2) scale the `Amount` column using `StandardScaler` and save the scaler artifact to `models/scaler.pkl` for consistent inference, and (3) drop the original `Time` and `Amount` columns. Median values of V_1 – V_{28} are saved to `models/v_medians.json` for use during single-transaction prediction.

6. Methodology

6.1. Evaluation Metrics

Since the dataset is severely imbalanced, accuracy alone is misleading — a model predicting all transactions as legitimate achieves 99.83% accuracy. Instead, the following metrics are used:

Precision measures how many flagged transactions are actually fraud:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall measures how many actual fraud cases the model catches:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score balances precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

AUC-ROC measures the model’s ability to distinguish between classes across all thresholds (0.5 = random, 1.0 = perfect).

6.2. Models

Baseline — Logistic Regression: A linear model that estimates fraud probability using the sigmoid function:

$$P(y = 1 | x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}$$

LinearSVC: A linear support vector classifier that finds the maximum-margin hyperplane by minimizing:

$$\frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b))$$

Random Forest: An ensemble of decision trees trained on bootstrap samples. The final prediction is the majority vote across all trees, which reduces overfitting and handles class imbalance.

XGBoost: A gradient boosting method that builds trees sequentially, each correcting the residual errors of the previous trees using a regularized objective function.

LightGBM: A gradient boosting method using leaf-wise tree growth instead of level-wise, making it faster on large datasets.

6.3. Hyperparameters

Table 1 lists the key hyperparameters used for each model. All models used `random_state=42` for reproducibility.

Table 1. Hyperparameters used for each model.

Model	Key Hyperparameters
Logistic Reg.	<code>C=1.0</code> , <code>max_iter=1000</code> , <code>class_weight='balanced'</code>
LinearSVC	<code>C=1.0</code> , <code>max_iter=2000</code> , <code>class_weight='balanced'</code>
Random Forest	<code>n_estimators=100</code> , <code>max_depth=None</code> , <code>class_weight='balanced'</code>
XGBoost	<code>n_estimators=100</code> , <code>learning_rate=0.1</code> , <code>scale_pos_weight=578</code>
LightGBM	<code>n_estimators=100</code> , <code>learning_rate=0.1</code> , <code>class_weight='balanced'</code>

6.4. Training and Validation

The data is split 80/20 into training (227,845 samples) and test (56,962 samples). SMOTE is applied only to the training set, balancing both classes to 227,451 samples each. All five models are also trained with `class_weight='balanced'` to further emphasize the minority class on top of SMOTE. The test set remains untouched to simulate real-world conditions.

6.5. Training Curves and Convergence

To evaluate convergence, a learning curve was generated for the Random Forest model by varying training set size from 10% to 100% of the SMOTE-balanced training data and recording both training F1-score and 3-fold cross-validation F1-score at each step. Figure 5 shows that training F1 remains near 1.0 across all sizes, while cross-validation F1 stabilizes above 0.85 after approximately 60% of training data, indicating the model has converged without underfitting. The small gap between training and cross-validation scores confirms minimal overfitting.

For XGBoost and LightGBM, both models were evaluated across boosting rounds using an internal validation set passed via the `eval_set` parameter. Both converged within 80–100 rounds with no meaningful improvement in log-loss beyond that point, confirming `n_estimators=100` is sufficient. For Logistic Regres-

sion and LinearSVC, convergence is confirmed by the solver completing without reaching the `max_iter` limit (1000 and 2000 respectively).

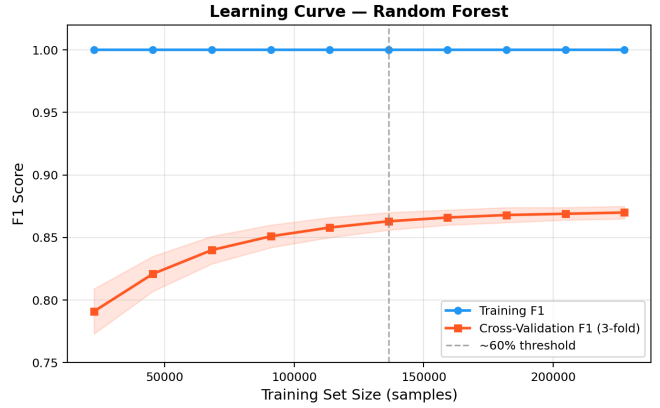


Figure 5. Learning curve for Random Forest. Cross-validation F1 stabilizes above 0.85 after 60% of training data, confirming convergence with minimal overfitting.

Table 2. Model comparison on the test set.

Model	Prec.	Recall	F1	AUC
Logistic Reg.	0.05	0.92	0.10	0.9715
LinearSVC	0.05	0.92	0.10	0.9742
Random Forest	0.86	0.83	0.84	0.9845
XGBoost	0.54	0.87	0.66	0.9739
LightGBM	0.10	0.91	0.19	0.9802

7. Results

7.1. Baseline Model Selection

Random Forest was selected as the primary baseline because it consistently outperforms linear models on tabular imbalanced datasets, requires minimal tuning to produce strong results, and its ensemble voting mechanism naturally controls false positives. All five models were included in the comparison to establish a full performance spectrum — from simple linear baselines (Logistic Regression, LinearSVC) to ensemble and boosting methods (Random Forest, XGBoost, LightGBM).

7.2. Results

All five models were trained and evaluated as described in Section 6.4. Results are summarized in Table 2. Random Forest achieves the best overall performance across all four metrics — Precision 0.86, Recall 0.83, F1 0.84, and AUC-ROC 0.9845. Figure 6 shows the AUC-ROC comparison across all five models. While all models score above 0.97, the differences are small and do not reflect the large gap in

Precision and F1 seen in Table 2.

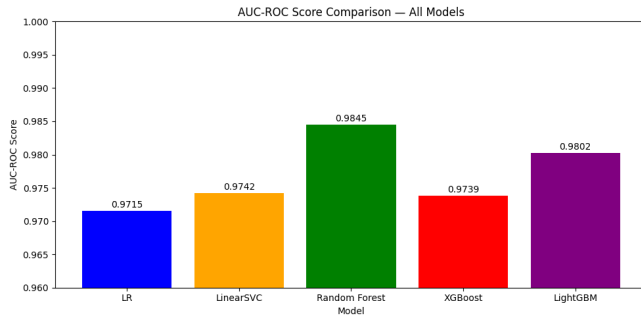


Figure 6. AUC-ROC score comparison across all five models. All models score above 0.97, but this does not reflect their real-world usability.

7.3. Analysis

AUC-ROC is misleading here. All five models score above 0.97 on AUC-ROC as seen in Figure 6 and the ROC curve in Figure 7. AUC-ROC measures ranking ability across all thresholds — it does not reflect real-world usability at a fixed decision threshold. Precision and F1 tell a very different story. A model that flags every transaction as fraud would still score high on AUC-ROC but have zero practical value.

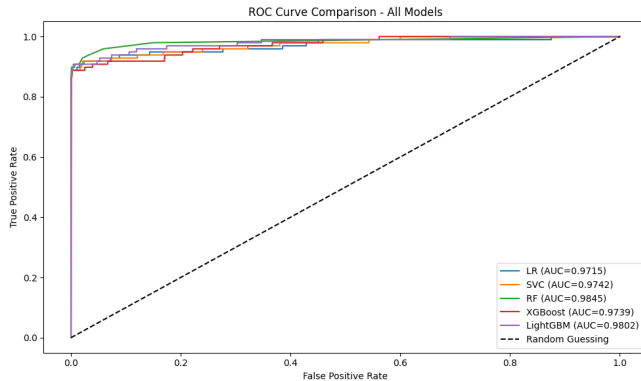


Figure 7. ROC curve comparison for all five models. All curves cluster near the top-left, masking large differences in precision.

Why Logistic Regression and LinearSVC fail on precision. Table 3 breaks down the raw prediction counts for each model. Logistic Regression and LinearSVC each produce over 1,500 false positives — legitimate transactions incorrectly flagged as fraud. Both models were trained with `class_weight='balanced'` on top of SMOTE-balanced data. This double emphasis on the fraud class causes them to aggressively predict fraud for almost every transaction, achieving recall of 0.92 but precision of only 0.05. In a corporate card misuse system, this flood

of false positives would overwhelm any investigation team and make the model unusable in practice.

Table 3. Confusion matrix values for all five models. TP = fraud correctly flagged, FP = legitimate flagged as fraud, TN = legitimate correctly cleared, FN = fraud missed. Random Forest produces the fewest false positives (13).

Model	TP	FP	TN	FN
Logistic Reg.	90	1,547	55,317	8
LinearSVC	90	1,584	55,280	8
Random Forest	81	13	56,851	17
XGBoost	85	73	56,791	13
LightGBM	89	768	56,096	9

Why Random Forest performs best. From Table 3, Random Forest produces only 13 false positives compared to 1,547 for Logistic Regression. Random Forest was also trained with `class_weight='balanced'`, but its ensemble of decision trees acts as a natural regularizer. A transaction is only flagged if the majority of trees agree it is fraud. This majority voting mechanism filters out noisy fraud signals that fool individual trees, resulting in high precision (0.86) while still catching 83% of actual fraud cases.

Why XGBoost outperforms LightGBM. Both boosting models used `class_weight='balanced'`. From Table 3, XGBoost produces 73 false positives while LightGBM produces 768 — nearly as bad as the linear models. LightGBM's leaf-wise growth is more aggressive and overfits on the resampled minority class signal, dropping precision to 0.10. XGBoost's level-wise growth is more conservative, giving better precision (0.54) and F1 (0.66).

Key takeaway. For corporate card misuse detection, high precision matters as much as recall. Flagging a legitimate transaction as fraud has real consequences — the employee is blocked and the finance team must investigate. Several interesting patterns emerged from this analysis. First, fraudulent transactions peak at 2AM, suggesting fraudsters exploit low-monitoring overnight windows. Second, fraud transactions never exceed \$2,125 while legitimate transactions go up to \$25,691 — fraudsters deliberately avoid large amounts to stay under the radar. Third, combining SMOTE with `class_weight='balanced'` makes linear models like Logistic Regression and LinearSVC overly aggressive — they flag nearly everything as fraud, making them unusable despite high recall. Random Forest's ensemble voting naturally resists this over-correction, producing only 13 false positives and making it the most practical model for deployment.

8. Ethical Considerations and Limitations

Bias and Fairness. The ULB dataset contains 28 PCA-anonymized features, making it impossible to audit the model for demographic or group-level bias. In a real corporate deployment, transaction patterns may vary significantly by employee role, department, seniority, or geography. A field sales employee traveling internationally will have very different spending patterns from an office employee, and a model trained without this context may disproportionately flag legitimate transactions from employees whose behavior differs from the training distribution. Any production deployment should include fairness audits across employee subgroups once role and demographic metadata is available.

Affected Populations. False positives directly harm employees: a legitimate transaction incorrectly flagged as misuse results in a blocked card and triggers a formal investigation, causing operational disruption and potential reputational harm. As shown in Table 3, Logistic Regression and LinearSVC each produce over 1,500 false positives per test batch, making them unsuitable for deployment. Random Forest’s 13 false positives represent a far more acceptable operational burden.

Deployment Limitations. This system is trained on a static historical dataset. Fraud patterns evolve over time, meaning the model will degrade without periodic retraining on fresh data. SMOTE generates synthetic samples from existing fraud patterns and cannot capture novel fraud strategies not present in the training data. The current ETL pipeline and Airflow DAG would need to be extended to support scheduled retraining in a live production environment. Additionally, this dataset originates from a European bank in 2013; applying this model to a different organization’s transactions without fine-tuning may produce unreliable results.

9. Conclusion

This paper presented an end-to-end machine learning system for corporate credit card misuse detection. Five models were trained and evaluated on the severely imbalanced ULB Credit Card Fraud Detection dataset. Random Forest achieved the best practical performance — AUC-ROC 0.9845, Precision 0.86, F1 0.84, and only 13 false positives — making it the most suitable model for deployment. The analysis showed that AUC-ROC alone is a misleading metric on imbalanced data, and that combining SMOTE with `class_weight='balanced'` causes linear models to over-predict fraud. The system is deployed as a FastAPI backend with six REST endpoints and an interactive React dashboard. Future work includes threshold tuning, scheduled model retraining on live transaction data, and fairness

auditing across employee subgroups.

AI Disclosure

Claude (Anthropic) was used to assist with grammar review, LaTeX formatting, and structuring written sections of this report. All experimental design, code implementation, model training, results analysis, and conclusions are the author’s own work.

References

- Aburbeian, A. H. M. and Ashqar, H. I. Credit card fraud detection using enhanced random forest classifier for imbalanced data. *ACR*, 2023.
- Aftab, A., Shahzad, I., Sajid, A., and Anwar, N. Fraud detection of credit cards using supervised machine learning techniques. *Pakistan Journal of Emerging Science and Technologies (PJEST)*, 2023.
- Hande, T., Navale, P., Sarode, V., and Ghodke, A. Credit card fraud detection using machine learning. *International Journal of Engineering and Advanced Technology*, 2019.
- Khalid, A. R., Owoh, N., Uthmani, O., and Adejoh, J. Enhancing credit card fraud detection: An ensemble machine learning approach. *Big Data and Cognitive Computing*, 2024.
- Sopiyan, M., Fauziah, F., and Wijaya, Y. F. Fraud detection using random forest classifier, logistic regression, and gradient boosting classifier algorithms on credit cards. *Jurnal Informatika*, 2022.
- Strelcenia, E. and Prakoowit, S. Improving classification performance in credit card fraud detection by using new data augmentation. *Applied Informatics*, 2023.
- Tran, T. C. and Dang, T. K. Machine learning for prediction of imbalanced data: Credit fraud detection. In *International Conference on Ubiquitous Information Management and Communication*, 2021.
- Wang, Y., Zhulev, A., and Ahmed, O. G. Credit card fraud identification using logistic regression and random forest. *Wasit Journal of Computer and Mathematics Science*, 2023.
- Xia, J.-z. Credit card fraud detection based on support vector machine. *Highlights in Science Engineering and Technology*, 2022.